PROJECT SCOPE DOCUMENT

Integrated Web-Based RISC-V Simulator and Computer Architecture Design Platform

A browser-based simulator with compiler-explorer functionality, custom ISA tools, processor design, and architecture analysis.

C Kalki Suvarn (ES24BTECH11011)
C Hemish Reddy (ES24BTECH11012)

1 Project Description

The objective of this project is to build an integrated web-based computer architecture simulator. The base experience is inspired by a browser-based RISC-V simulator (RISC-V Simulator). It will be extended with compiler and side-by-side output comparison features inspired by Compiler Explorer, and a visual Processor Design Studio inspired by tools such as Logisim Evolution and CircuitVerse.

The platform aims to connect source code, assembly, machine-code encoding, ISA definitions, processor execution and architectural analysis in one environment. Students should be able to write or load a program, inspect its instructions and machine state, compare compiler output, experiment with a custom ISA or supported processor design, and study cache and branch-prediction behaviour.

The core scope includes the web simulator, compiler/cross-compilation views, assembly-to-C-like reconstruction, custom ISA design, instruction encoding/decoding, processor design, memory exploration, cache analysis and branch prediction. FPGA/RTL simulation, virtual embedded boards and expanded account/course functionality are stretch goals.

2 Must-Have Goals

ID	Goal	Description
M1	Working web simulator	Browser-based RISC-V assembly editor and execution; run, reset and step through instructions; inspect registers, PC and memory; show errors and provide example programs.
M2	Compiler comparison	Compiler Explorer-style interface for supported compilers, targets and optimization flags; display source alongside generated assembly and machine code.
M3	Code translation and graph	Show assembly and reconstructed C-like code side by side; display a control-flow graph and highlight the current instruction/statement/node during execution when mapping is available.
M4	Custom ISA Designer	Define instruction formats, opcodes, register/immediate fields and aliases; validate conflicts; save/load ISA definitions; generate encoder/decoder and assembler support for the supported subset.
M5	Instruction encoder/decoder	Convert supported assembly to binary/hex and decode machine code into fields and readable instructions; support standard RISC-V formats and defined custom formats.
M6	Processor Design Studio	Visual datapath editor with reusable and nested components, wires, ports and signal widths; distinguish combinational/sequential logic; validate designs and execute a defined subset.
M7	Memory explorer	Inspect registers and memory regions/values in suitable numeric formats; step through execution and observe PC, stack pointer and memory changes.
M8	Memory hierarchy/cache lab	Trace-driven cache model with supported mapping/associativity options; show tag/index/offset, hits, misses, evictions, rates and documented simulated access costs.
M9	Branch prediction lab	Trace-driven static branch prediction; show predicted and actual outcomes, correct predictions, mispredictions, accuracy and configured penalties where modelled.
M10	Integrated workflow and tests	Connect the modules through shared artifacts and execution events; provide unit, regression and end-to-end tests, documentation and a reproducible demonstration.

Stretch Goals

These goals will be attempted only after the must-have goals have been completed and integrated.

ID	Goal	Description
S1	FPGA / RTL sandbox	Run supported Verilog/SystemVerilog designs in an isolated simulator; execute testbenches and inspect waveforms. RTL generation from supported processor designs, synthesis and physical FPGA deployment depend on feasibility and hardware access.
S2	Virtual embedded boards	Add a virtual board and code editor with interactive LEDs, buttons, switches, timers and UART; extend to ESP32, Raspberry Pi Pico, STM32 or a virtual RISC-V board where feasible.
S3	Advanced processor features	Expand the processor designer to more complex pipelines, forwarding, hazard detection, timing estimates and broader automatic RTL generation.
S4	Advanced predictors and caches	Add dynamic predictors (one-bit/two-bit, local/global or tournament), multilevel caches, additional replacement/write policies and direct use of processor-generated traces.
S5	Advanced compiler support	Add more compiler targets, LLVM IR, richer optimization comparisons, improved assembly-to-C-like reconstruction and a dedicated compiler backend for custom ISAs.
S6	Workspace and course tools	Add cloud persistence, accounts, project sharing/versioning, instructor-managed courses, assignments, submission history and automated grading.

3 Phases and Approach

Stage	Planned Work
Foundation and baseline	Review the reference RISC-V simulator and Compiler Explorer workflow. Confirm supported instructions and toolchains, establish the repository/build process, and record baseline behaviour.
Web simulator and compiler views	Implement the core editor, execution controls, register/memory inspection, compiler-output comparison, and supported source/assembly views.
ISA and processor design	Implement custom ISA definitions, encoding/decoding, assembler support and a defined executable subset of the visual processor designer.
Architecture analysis	Implement the trace-driven cache and branch-prediction labs; define metrics and validate against hand-calculated/reference cases.
Integration and verification	Connect modules through shared artifact and trace formats. Run unit, regression and end-to-end tests; resolve discrepancies.
Finalization	Document supported features and limitations, prepare setup instructions, report, and final demonstration. Attempt stretch goals only after must-have completion.

4 Anticipated Risks and Mitigation

Risk	Mitigation
Broad scope and integration complexity	Keep the must-have boundary explicit, agree shared interfaces early, and integrate modules continuously.
Custom ISA/compiler complexity	Separate ISA definition, assembler, reference execution and compiler-backend responsibilities. A custom ISA requires a compatible backend or restricted-language compiler.
Assembly-to-C reconstruction limits	Label output as reconstructed C-like code; validate behaviour rather than expecting original source recovery.
Incorrect simulator or architectural metrics	Use deterministic unit/reference tests, hand-verified traces and documented timing assumptions.
Processor designer complexity	Define component ports and execution semantics; begin with a clearly supported subset and expand it.
Stretch goals consuming core time	Attempt FPGA/RTL, virtual boards and advanced workspace features only after the must-have modules are functional.

5 Validation and Final Deliverables

• Test instruction encoding/decoding and supported ISA execution against known results.

• Validate register and memory effects, processor components and supported processor execution.

• Use identical workloads when comparing compiler outputs, caches and branch predictors.

• Report simulated instruction counts, cycles, CPI/IPC where a cycle model exists, cache statistics and branch-prediction metrics; label assumptions clearly.

• Deliver source code, tests, setup instructions, README, architecture notes, final report, and an end-to-end demonstration.

Project Scope: Success Criteria and LLM Disclosure

6 Definition of Success

The project will be considered successful when the must-have web simulator, compiler views, custom ISA tools, Processor Design Studio, cache lab and branch-prediction lab work meaningfully and are integrated into a coherent platform.

The final demonstration should show:

- Code execution through the web simulator.
- Inspection of machine state, including registers, the program counter and memory.
- Side-by-side compiler and assembly comparison.
- Computer architecture analysis using the cache and branch-prediction labs.

FPGA/RTL support, virtual boards and expanded course or workspace features are stretch goals. They will be attempted only after the must-have features are functional and integrated.

7 LLM Assistance Disclosure

LLM tools were used for document generation and formatting, summarizing reference documents, and generating boilerplate code.

The project architecture, core implementation, integration, testing, validation and final technical decisions will be carried out and verified by the project team.